

GIẢI THUẬT TÌM KIẾM

1. Nhu cầu Tìm kiếm và Sắp xếp

- Trong thực tế, khai thác dữ liệu hầu như lúc nào cũng phải thực hiện thao tác tìm kiếm.
- Việc tìm kiếm nhanh hay chậm tùy thuộc vào **trạng thái và trật tự của dữ liệu** trên đó.
- Để tìm kiếm dữ liệu dễ dàng và nhanh chóng, trước khi thao tác thì dữ liệu trên mảng và tập tin đã **có thứ tự**.
- Thao tác sắp xếp dữ liệu là một trong những thao tác cần thiết.

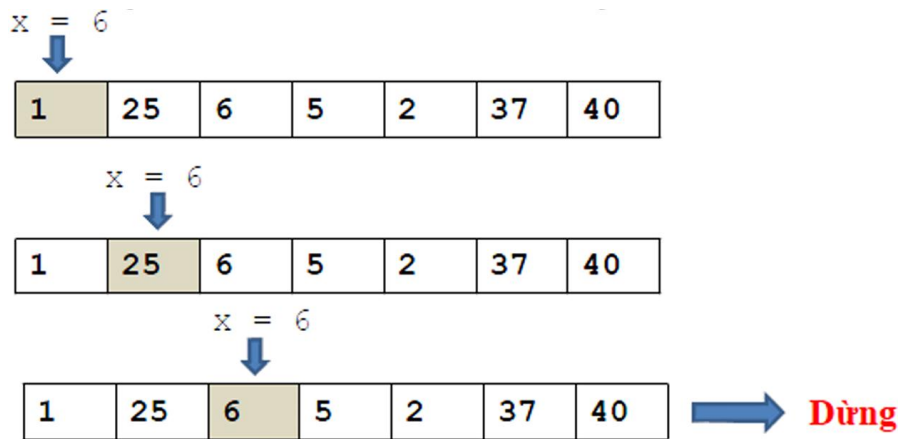
2. Bài toán tìm kiếm

- Tìm kiếm là thuật toán tìm 1 phần tử có giá trị cho trước trong một tập các phần tử.
- Cho một dãy gồm n bản ghi $r[1 \dots n]$. Mỗi bản ghi $r[i]$ tương ứng với một khóa $k[i]$. Hãy tìm bản ghi có khóa X cho trước. X được gọi là khóa tìm kiếm hay đối trị tìm kiếm (argument).
- Công việc tìm kiếm sẽ hoàn thành nếu như có một trong hai tình huống sau xảy ra:
 - *Tìm được bản ghi có khóa tương ứng bằng X , lúc đó phép tìm kiếm **thành công**;*
 - *Không tìm được bản ghi nào có khóa bằng X cả, phép tìm kiếm **thất bại**.*
- Để đơn giản cho việc trình bày giải thuật thì bài toán có thể phát biểu như sau: Cho mảng $A[n]$, hãy tìm trong A phần tử có khóa bằng X .

3. Thuật toán tìm kiếm tuyến tính (Linear Search)

3.1. Linear Search on Unordered Array/ Tìm kiếm tuần tự vét cạn (Exhaustive Linear)

- **Ý tưởng:** So sánh X lần lượt với phần tử thứ 1, thứ 2,... của mảng A cho đến khi gặp được khóa cần tìm, hoặc tìm hết mảng mà không thấy.
- **Thuật toán:**
 - **Input:** Danh sách A có n phần tử, giá trị khóa x cần tìm.
 - **Output:** Chỉ số i của phần tử a_i trong A có giá trị khóa là x . Trong trường hợp không tìm thấy $i = -1$
- **Các bước tiến hành**
 - Bước 1: Khởi gán $index=0$;
 - Bước 2: So sánh $a[index]$ với giá trị x cần tìm, có 2 khả năng
 - $a[index] == value$ tìm thấy value. Dừng; // return index;
 - $a[index] != value$ sang bước 3;
 - Bước 3: $index=index+1$ // Xét tiếp phần tử kế tiếp trong mảng
 - Nếu $index==N$: Hết mảng. Dừng; // return -1;
 - Ngược lại: Lặp lại bước 2;
- **Ví dụ:** $A = \{1, 25, 6, 5, 2, 37, 40\}$, $x = 6$



3.2. Ordered Linear Search

- Search an **ordered array** of integers for a value and return its index if the value is found; Otherwise, return -1.
- Linear search can stop immediately **when it has passed the possible position of the search value**.

```
int lsearch(int data[],    // input: array
            int size,     // input: array size
            int value) {  // input: value to find
                        // output: index if found
    for (int index = 0; index < size; index++) {
        if (data[index] > value) // cho trường hợp ordered array
            return -1;
        else if (data[index] == value)
            return index;
    }
    return -1;
}
```

- Số phép so sánh của thuật toán trong trường hợp **xấu nhất** (phần tử cần tìm không có trong mảng hoặc nằm ở vị trí cuối) là **$2n$** . Vì ở mỗi lần duyệt qua một phần tử trong mảng, ta cần thực hiện 2 điều kiện (cũng là 2 phép so sánh):
 - **Điều kiện 1:** Phần tử hiện tại có bằng giá trị cần tìm không ($A[i] == x$).
 - **Điều kiện 2:** Đã đi hết mảng chưa ($i < n$).
- Để giảm thiểu số phép so sánh trong vòng lặp cho thuật toán, ta thêm phần tử "lính canh" vào cuối dãy.

3.3. Thuật toán tìm kiếm tuyến tính cải tiến - Tìm kiếm tuần tự lính canh (Sentinel Linear Search)

- Trong **Sentinel Linear Search**, một giá trị "lính canh" (sentinel) được thêm vào cuối mảng trước khi tìm kiếm. Giá trị này chính là giá trị cần tìm.
- Điều này đảm bảo rằng thuật toán luôn tìm thấy giá trị cần tìm (dù ở vị trí gốc trong mảng hay tại vị trí lính canh), giúp **loại bỏ việc kiểm tra điều kiện "đã đi hết mảng" trong mỗi lần lặp**.
- Trong vòng lặp của Sentinel Linear Search, chỉ cần thực hiện **một phép so sánh** ($A[i] == x$) cho mỗi phần tử, thay vì hai phép như Linear Search thông thường.

- **Ý tưởng:**
 - Lính canh là phần tử có giá trị bằng với phần tử cần tìm và đặt ở cuối mảng.
 - Thêm phần tử cảm canh a[n] có khóa x vào A, khi này A có n+1 phần tử.
 - **Chỉ cần điều kiện dừng là tìm thấy phần tử a[i] có khóa x.**
- **Các bước tiến hành:**
 - **B1:**
 - Khởi gán i=0;
 - Khởi gán a[n]=x; // a[n] là phần tử cảm canh
 - **B2:** So sánh a[i] với giá trị khóa x cần tìm, có 2 khả năng:
 - (a[i]==x): sang bước 4;
 - (a[i]!=x): sang bước 3;
 - **B3:** i=i+1; // Xét phần tử kế tiếp trong mảng
 - **B4:**
 - Nếu i < n: tìm thấy x. Dừng
 - Nếu i==n: vị trí tìm thấy là vị trí của phần tử cảm canh → không tìm thấy x. Dừng.

```
int SequentialSearch(int A[], int n, int x)
{
    int i=0, A[n]=x;
    //A[n] là phần tử lính canh
    while(A[i]!=x)
        i++;
    if(i==n) return 0; //Tìm không thấy x
    else return 1; //Tìm thấy
}
```

```
int SequentialSearch(int arr[], int n, int x)
{
    int i = 0;
    arr[n] = x; // phần tử cảm canh
    while (arr[i] != x)
        i++;
    if (i < n)
        return i;
    else // i == n
        return -1;
}
```

3.4. Độ phức tạp

- Trường hợp tốt nhất: O(1)
- Trường hợp tồi nhất: O(n)
- Trường hợp trung bình: O(n)

4. Thuật toán tìm kiếm nhị phân - Binary Search

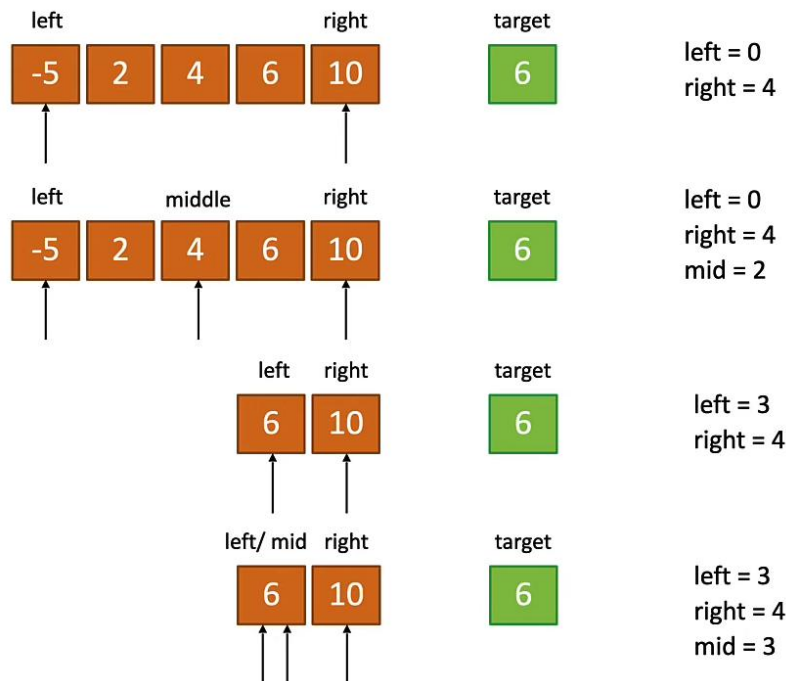
- Binary search được áp dụng trên **mảng đã có thứ tự**.
- *Binary search dựa trên chiến lược “Chia để trị” (divide and conquer):*
 - **Bắt đầu tìm kiếm tại phần tử giữa của mảng.**
 - Nếu giá trị tại phần tử giữa bằng giá trị cần tìm thì **kết thúc**.
 - Nếu giá trị tại **phần tử giữa nhỏ hơn giá trị cần tìm** thì loại bỏ nửa đầu tiên của mảng trong lần xét kế tiếp.
 - Nếu giá trị tại **phần tử giữa lớn hơn giá trị cần tìm** thì loại bỏ nửa sau của mảng trong lần xét kế tiếp.
 - Lặp lại quá trình cho tới khi **phần tử được tìm thấy**, hoặc **cho tới khi toàn bộ mảng được loại bỏ**.

- **Thuật toán:**

- **Input:** Danh sách A có n phần tử **đã có thứ tự $\mathfrak{R}$** , giá trị khóa x cần tìm.
- **Output:** Chỉ số i của phần tử a_i trong A có giá trị khóa là x . Trong trường hợp không tìm thấy $i = -1$.
- Giả sử dãy tìm kiếm hiện hành bao gồm các phần tử nằm trong $a_{\text{first}}, a_{\text{last}}$, các bước của giải thuật như sau:

- **Bước 1:** $\text{first} = 0; \text{last} = N-1$;
- **Bước 2:**
 - $\text{middle} = (\text{first} + \text{last})/2$; // chỉ số phần tử giữa dãy hiện hành
 - So sánh $a[\text{middle}]$ với value. Có 3 khả năng:
 - $a[\text{middle}] = \text{value}$: tìm thấy. Dừng; // return middle
 - $a[\text{middle}] > \text{value}$: $\text{last} = \text{middle} - 1$;
 - $a[\text{middle}] < \text{value}$: $\text{first} = \text{middle} + 1$;
- **Bước 3:**
 - Nếu $\text{first} \leq \text{last} \rightarrow$ còn phần tử trong dãy hiện hành $\rightarrow$ Lặp lại bước 2
 - Ngược lại: Dừng; // return -1;

- **Ví dụ:**



- **Độ phức tạp thuật toán:**

- Trường hợp xấu nhất: $O(\log(n))$
- Trung bình: $O(\log(n))$
- Trường hợp tốt nhất: $O(1)$